چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۲۴

CI/CD و DevOps — بخش دوم: امضا، خودکارسازی انتشار، rollout و rollback



اسماعیل مرادی

@esmaeeil-moradi700

چرا انتشار در ۲۰۲۶ مهم است؟

روز ۲۳ یک commit را به یک build سبز و تست‌شده رساند. روز ۲۴ کیلومتر آخر است: یک **نسخهٔ امضاشده روی دستگاه کاربر** — امن، تکرارپذیر، و با راه نجات وقتی چیزی خراب می‌شود.

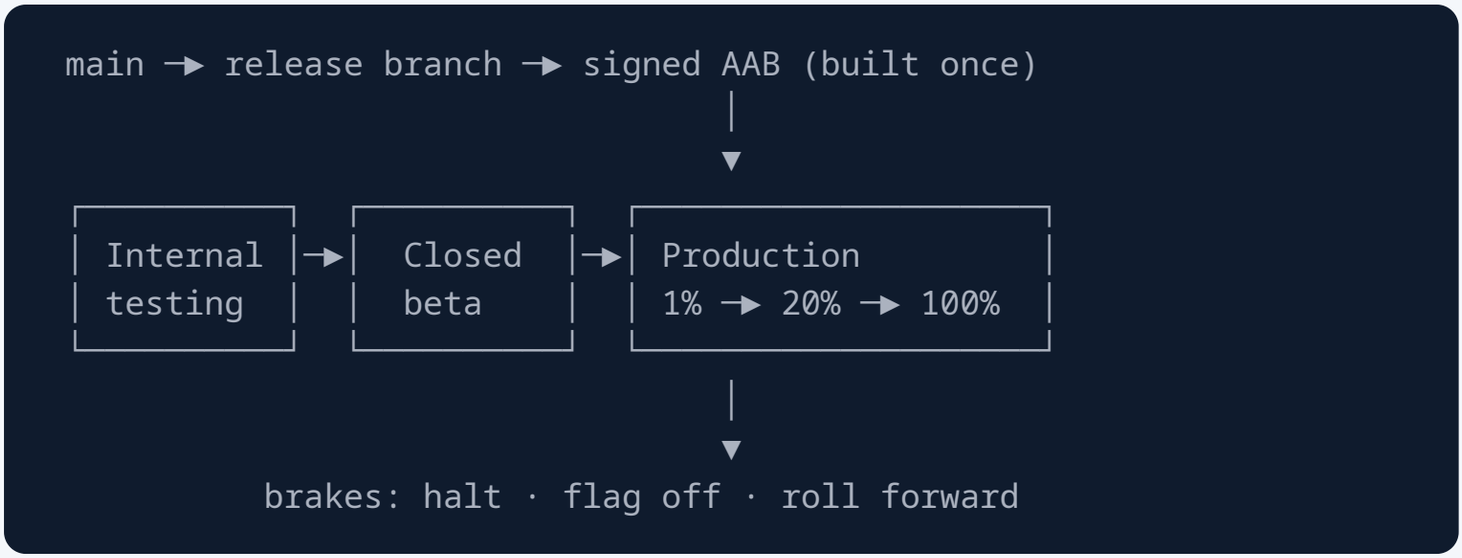
- کلیدهای امضا هویت اپ شما هستند — اگر یکی لو برود یا گم شود، ممکن است به‌روزرسانی‌ها متوقف شوند
- انتشار باید کسل‌کننده باشد: **یک بار build، ارتقای همان artifact**، آپلود خودکار
- باینری منتشرشده پس‌گرفتنی نیست: **rollout مرحله‌ای، halt و فلگ‌ها** ترمزهای سریع‌اند
- **in-app update** و گیت حداقل نسخه، کاربران را از نسخه‌های معیوب جدا می‌کنند
- **trunk-based development** و **release train** ریتم انتشار را قابل پیش‌بینی می‌کنند

سطح	سؤال‌ها
جونیور	کلید آپلود در برابر کلید امضای اپ، versionCode، trackهای تست Play
مید	امضا در CI، خودکارسازی انتشار، rollout مرحله‌ای، in-app update
سینیور	remote config و kill switch، «برش گردان!»، secret بدون کلید بلندمدت
آرکیتکت	trunk-based development، release train، پایپ‌لاین انتشار با دسترس‌پذیری بالا



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: یک بار build، ارتقا، افزایش تدریجی



- **یک artifact:** همان AABی که از internal testing رد شد، به production می‌رسد
- **افزایش بر اساس داده:** هر پله منتظر شاخص‌های کرش، ANR و کسب‌وکار می‌ماند
- **دکمهٔ undo وجود ندارد:** جلوی آسیب را می‌گیرید و رو به *جلو* درستش می‌کنید

> **ایدهٔ واحد پشت هر انتشار**
>
> deploy کردن یک باینری و release کردن یک فیچر دو رویداد جدا هستند. جدا نگهشان دارید تا هر انتشار یک ترمز داشته باشد.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

کلید آپلود در برابر کلید امضای اپ، versionCode و versionName، trackهای تست Play



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. دو کلید در Play App Signing

کلید	نزد چه کسی	چه چیزی را امضا می‌کند	اگر گم شود یا لو برود
کلید امضای اپ	گوگل، داخل Play App Signing	APKهایی که Play از AAB شما می‌سازد و به کاربر می‌دهد	نزد گوگل می‌ماند؛ ارتقای کلید ممکن است اما رویدادی نادر و برنامه‌ریزی‌شده است
کلید آپلود	شما و CI	AABی که آپلود می‌کنید؛ گوگل آن را وارسی و دوباره امضا می‌کند	مالک حساب در Play Console **ریست کلید آپلود** را درخواست می‌کند

- برای اپ‌های جدید از وقتی App Bundle برایشان اجباری شد (اوت ۲۰۲۱) الزامی است: Play برای هر دستگاه APK مخصوص می‌سازد، پس خود Play باید آن را امضا کند
- buildهای دیباگ کلید سومی دارند — debug keystore که خودکار ساخته می‌شود



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«*Google Sign-In و Maps در APK نسخهٔ release من کار می‌کنند، اما وقتی از Play نصب می‌شود نه.*» — کاربران APKهایی می‌گیرند که با **کلید امضای اپ** امضا شده‌اند؛ پس اثر انگشت SHA-1 و SHA-256 همین کلید (در Play Console، بخش App integrity) باید در Firebase، Google Cloud و فایل `assetlinks.json` ثبت شود. ثبت فقط اثر انگشت کلید آپلود یا دیباگ، باگ کلاسیک است.

تلهٔ مصاحبه

«*keystore گم شود، اپ برای همیشه از دست رفته.*» — برای مدل قدیمی که خودتان امضا می‌کردید درست است. با Play App Signing، کلید آپلودِ گم‌شده با ریست قابل بازیابی است. با این حال یک نسخهٔ پشتیبان آفلاین نگه دارید: ریست فوری نیست.

سؤال تکمیلی

«*ما در فروشگاه‌های دیگر هم منتشر می‌کنیم.*» — اندروید فقط به‌روزرسانی‌ای را نصب می‌کند که با **همان** کلید امضا شده باشد. یا هنگام ثبت‌نام در Play App Signing کلید امضای خودتان را به Play بدهید و همه‌جا از همان استفاده کنید، یا بپذیرید که کاربران بدون نصب دوباره نمی‌توانند بین فروشگاه‌ها جابه‌جا شوند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. versionCode در برابر versionName

- **`versionCode`** — یک عدد صحیح مثبت. هر آپلود برای یک اپ به عددی جدید و بزرگ‌تر نیاز دارد؛ سقف Play عدد ۲۱۰۰۰۰۰۰۰۰ است
- **`versionName`** — رشته‌ای که کاربر می‌بیند، مثل `5.2.1`؛ قاعده‌ای ندارد
- اندروید versionCode کوچک‌تر را روی بزرگ‌تر نصب نمی‌کند — به همین دلیل در موبایل **rollback وجود ندارد** (سؤال ۹)

```
// app/build.gradle.kts
android {
    defaultConfig {
        versionCode = providers
            .environmentVariable("VERSION_CODE")
            .map(String::toInt)
            .getOrElse(1)          // local builds
        versionName = "5.2.1"
    }
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. عدد از کجا می‌آید؟

تلهٔ مصاحبه

«*versionCode را با* `git rev-list --count HEAD` *بسازیم.*» — اکشن `actions/checkout` به‌طور پیش‌فرض فقط یک commit را fetch می‌کند، پس نتیجه روی CI همیشه **۱** است. باید `fetch-depth: 0` بدهید — و باز هم بعد از بازنویسی تاریخچه خراب می‌شود.

تلهٔ مصاحبه

«*از* `GITHUB_RUN_NUMBER` *استفاده کن.*» — خوب است تا وقتی که یک فایل workflow جدید یا مهاجرت به CI دیگر، شمارش را از ۱ شروع کند. یک offset ثابت اضافه کنید، یا بالاترین کد استفاده‌شده را از Play بپرسید؛ Gradle Play Publisher و fastlane این کار را می‌کنند.

سؤال تکمیلی

«*چرا semver را داخلش رمز نکنیم، مثلاً 5.2.1 به 50201؟*» — جایی برای build دوم از همان نسخه نمی‌ماند: RC2، hotfixای که دوباره build شده، آپلود دوباره بعد از رد شدن در بازبینی. versionCode مال CI است؛ versionName مال آدم‌ها.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. trackهای تست در Play

track	مخاطب	کاربرد
Internal testing	تا ۱۰۰ تستری که دعوت می‌کنید	تست دود هر release candidate؛ سریع در دسترس
Closed testing	فهرست‌های دعوت یا Google Groups؛ چند track ممکن است	بتا با کاربران واقعی
Open testing	هر کسی که از صفحهٔ فروشگاه عضو شود	بتای عمومی در مقیاس بزرگ
Production	همهٔ کاربران واجد شرایط، در صورت نیاز مرحله‌ای	انتشار واقعی

- **Internal app sharing:** یک build را با لینک به اشتراک بگذارید، بدون بالا بردن versionCode — راه تست in-app update هم همین است (سؤال ۷)
- حساب‌های توسعه‌دهندهٔ شخصیِ جدید قبل از دسترسی به production باید یک closed test را کامل کنند — قواعد فعلی را چک کنید

تلهٔ مصاحبه

«*برای production از روی تگ دوباره build بگیر.*» — آن‌وقت production باینری‌ای را اجرا می‌کند که هیچ‌کس تستش نکرده. AAB را **یک بار** بسازید و همان artifact را track به track **ارتقا دهید**.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

امضا در CI، خودکارسازی انتشار، rollout مرحله‌ای و halt، in-app update



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. workflow امضا در CI

```yaml
bundle:
  runs-on: ubuntu-latest
  environment: release       # secrets live here
  steps:
    # checkout, JDK, setup-gradle …
    - name: Restore upload key
      run: |
        echo "$KEYSTORE_B64" | base64 -d \
          > "$RUNNER_TEMP/upload.jks"
      env:
        KEYSTORE_B64: ${{ secrets.UPLOAD_KEYSTORE_B64 }}
    - run: ./gradlew bundleRelease
      env:
        SIGNING_STORE_FILE: ${{ runner.temp }}/upload.jks
        SIGNING_STORE_PASSWORD: ${{ secrets.STORE_PASSWORD }}
        SIGNING_KEY_ALIAS: upload
        SIGNING_KEY_PASSWORD: ${{ secrets.KEY_PASSWORD }}
```

- keystore یک فایل باینری است؛ پس آن را به شکل base64 در یک secret نگه دارید و در پوشهٔ موقت runner به فایل برگردانید
- secretهای داخل یک **environment** می‌توانند تأیید انسانی بخواهند و فقط به branchهای انتخاب‌شده محدود می‌شوند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. امضا در Gradle

```kotlin
// app/build.gradle.kts
fun env(name: String) =
    providers.environmentVariable(name).orNull

android {
    signingConfigs {
        create("release") {
            storeFile = env("SIGNING_STORE_FILE")
                ?.let { file(it) }
            storePassword = env("SIGNING_STORE_PASSWORD")
            keyAlias = env("SIGNING_KEY_ALIAS")
            keyPassword = env("SIGNING_KEY_PASSWORD")
        }
    }
    buildTypes {
        release {
            signingConfig = signingConfigs.getByName("release")
        }
    }
}
```

تلهٔ مصاحبه

keystore داخل git و پسوردها در `gradle.properties`، «چون ریپو خصوصی است». حالا هر clone، هر لپ‌تاپ و هر fork کلید آپلود شما را دارد. جای کلید در secretهای CI یا یک vault است — به‌علاوهٔ یک نسخهٔ پشتیبان آفلاین.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. خودکارسازی انتشار: دو ابزار، یک API

	Gradle Play Publisher	fastlane `supply`
کجا زندگی می‌کند	یک پلاگین Gradle — همان build، یک task برای هر variant	زنجیرهٔ ابزار Ruby؛ laneها در یک `Fastfile`
نقطهٔ قوت	runtime اضافه ندارد؛ متن فروشگاه و release notes در خود ریپو نسخه‌بندی می‌شوند	iOS و Android در یک ابزار؛ اسکرین‌شات؛ پلاگین‌ها
هزینه	باید با نسخه‌های AGP هم‌قدم بماند	Ruby و Bundler روی هر runner

- هر دو **Google Play Developer API** را صدا می‌زنند: یک *edit* باز می‌کنند، AAB را آپلود می‌کنند، release روی track را تنظیم و commit می‌کنند
- برای هر ریپو یکی را انتخاب کنید — دو ابزار که روی یک اپ منتشر می‌کنند همه را گیج می‌کنند

تلهٔ مصاحبه

«هر *merge* به `main` مستقیم به ۱۰۰ درصد *production* برود.» — این continuous deployment بدون ترمز است. آپلود به internal testing را خودکار کنید؛ ارتقا به production با یک انسان یا با jobی که به شاخص‌ها گره خورده است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. در کد

```
// app/build.gradle.kts
// plugins { id("com.github.triplet.play") }
play {
    track.set("internal")
    defaultToAppBundles.set(true)
}
// CI: ANDROID_PUBLISHER_CREDENTIALS holds the credentials
// ./gradlew publishReleaseBundle
```

```
# fastlane/Fastfile — promote the tested build, don't rebuild
lane :promote do
  upload_to_play_store(
    track: "internal",
    track_promote_to: "production",
    rollout: "0.05",
    skip_upload_aab: true,
    skip_upload_metadata: true
  )
end
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. rollout مرحله‌ای و halt

- **rollout مرحله‌ای** نسخه را به درصد تصادفی از کاربران واجد شرایط production عرضه می‌کند؛ پله‌پله بالا ببرید (مثلاً ۱، ۵، ۲۰، ۵۰ و بعد ۱۰۰ درصد)
- **halt** جلوی دریافت نسخه توسط کاربران جدید را می‌گیرد. کاربرانی که به‌روز کرده‌اند، **آن را نگه می‌دارند**
- بعد از halt: یا ادامهٔ rollout، یا یک اصلاح با **versionCode بزرگ‌تر** که جایگزینش شود
- در Play Developer API یک release با وضعیت `inProgress` و یک `userFraction` شروع می‌شود و بعد به `halted` یا `completed` می‌رسد

تلهٔ مصاحبه

«*halt کاربران را به نسخهٔ قبل برمی‌گرداند.*» — نه. فقط جلوی گسترش را می‌گیرد. کاربرانی که باگ را دارند به kill switch (سؤال ۸) یا نسخهٔ اصلاح‌شده (سؤال ۹) نیاز دارند.

تلهٔ مصاحبه

«*یک ساعت روی ۱ درصد، بعد ۱۰۰ درصد.*» — بعضی باگ‌ها روزها طول می‌کشد تا دیده شوند: migration روی داده‌های قدیمی، کار پس‌زمینه، دستگاه‌های کمیاب. هر پله را به **زمان و حجم** گره بزنید و با نسخهٔ قبل در همان سن مقایسه کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. اول ترمز را خودکار کنید

```
scheduled job (e.g., every 30 min)
        │
        ▼
vitals of the new versionCode
(Crashlytics, Play Developer Reporting API)
        │
        ▼
worse than the previous version?
  ├── no  → wait, or raise userFraction
  └── yes → set status "halted" via the
            Play Developer API, page on-call
```

- halt ارزان است و خودکار کردنش امن است؛ بالا بردن درصد باید به همان داده‌ها و در ابتدا به یک انسان وابسته باشد
- نرخ کرش و ANR را دنبال کنید — آستانه‌های رفتار بد Play (روز ۲۸) روی دیده‌شدن اپ در فروشگاه اثر می‌گذارند

نظر من

قبل از پدال گاز، ترمز را خودکار کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. حالت‌های flexible و immediate در in-app update

	Flexible	Immediate
تجربهٔ کاربر	دانلود در پس‌زمینه؛ کاربر به کارش ادامه می‌دهد	تمام‌صفحه؛ Play نصب و ری‌استارت می‌کند
برای	بیشتر به‌روزرسانی‌ها	اصلاحات حیاتی
کار شما	منتظر `DOWNLOADED` بمانید و بعد `completeUpdate()` را صدا بزنید	ادامهٔ جریان نیمه‌کاره در `onResume`

- کتابخانه: `com.google.android.play:app-update` و نسخهٔ `-ktx` آن؛ پیشرفت نصب از طریق یک `InstallStateUpdatedListener` می‌رسد
- سیگنال‌ها: `updatePriority()` که عددی از ۰ تا ۵ است و برای هر release **فقط** از طریق Play Developer API تنظیم می‌شود، و `clientVersionStalenessDays()`

تلهٔ مصاحبه

هنوز وابسته به کتابخانهٔ یکپارچهٔ قدیمی `com.google.android.play:core` . این کتابخانه به کتابخانه‌های جداگانه تقسیم شد — `app-update` ، `review` ، `feature-delivery` و... — و نسخهٔ قدیمی با targetSdkهای جدید سازگار نیست؛ مهاجرت کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. بررسی به‌روزرسانی حیاتی

```kotlin
private val updateLauncher = registerForActivityResult(
    ActivityResultContracts.StartIntentSenderForResult(),
) { result ->
    // RESULT_CANCELED = user declined; min-version gate decides
}

private fun checkForCriticalUpdate() {
    val manager = AppUpdateManagerFactory.create(this)
    manager.appUpdateInfo.addOnSuccessListener { info ->
        val critical = info.updateAvailability() ==
            UpdateAvailability.UPDATE_AVAILABLE &&
            info.updatePriority() >= 4 &&
            info.isUpdateTypeAllowed(AppUpdateType.IMMEDIATE)
        if (!critical) return@addOnSuccessListener
        manager.startUpdateFlowForResult(
            info, updateLauncher,
            AppUpdateOptions
                .defaultOptions(AppUpdateType.IMMEDIATE),
        )
    }
}
```

تلهٔ مصاحبه

«*immediate update کاربر را مجبور می‌کند.*» — کاربر می‌تواند لغو کند، و این API فقط برای نصب‌های Google Play کار می‌کند. اجبار واقعی با **گیت حداقل نسخه‌ای** است که خودتان می‌سازید (روی سرور یا remote config)، با لینک فروشگاه به‌عنوان راه جایگزین.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

remote config و kill switch، «برش گردان!»، secret بدون کلید بلندمدت



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. remote config و kill switch

```
val config = Firebase.remoteConfig
config.setConfigSettingsAsync(
    remoteConfigSettings {
        minimumFetchIntervalInSeconds =
            if (BuildConfig.DEBUG) 0L else 3600L
    },
)
// Safe values when nothing was ever fetched (offline start)
config.setDefaultsAsync(R.xml.remote_config_defaults)
config.fetchAndActivate()

// Real-time: react to a kill switch without waiting
config.addOnConfigUpdateListener(object : ConfigUpdateListener {
    override fun onUpdate(update: ConfigUpdate) {
        if ("payments_v2_enabled" in update.updatedKeys) {
            config.activate()
        }
    }
    override fun onError(e: FirebaseRemoteConfigException) {}
})
```

- حداقل فاصلهٔ پیش‌فرض fetch برابر ۱۲ ساعت است — برای buildهای دیباگ کمش کنید، نه کورکورانه در production
- فلگ‌های type-safe، مالک فلگ و آزمایش‌ها: روز ۲۹. امروز: فلگ به‌عنوان **ترمز انتشار**



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. طراحی kill switch

- kill switch فقط برای کدی وجود دارد که **همراه** چک منتشر شده — قبل از انتشار روی هر مسیر پرریسک یکی بگذارید (پرداخت، موتور sync جدید، SDK شخص ثالث)
- **مقدار پیش‌فرض** همان چیزی است که اولین اجرای آفلاین می‌بیند: فیچرهای جدید پیش‌فرض خاموش، و switchهای فیچرهای موجود پیش‌فرض روشن
- الان fetch کنید و **در یک نقطهٔ امن activate کنید** (اجرای بعدی، صفحهٔ بعدی)؛ listener بلادرنگ را فقط برای switchهای اضطراری به کار ببرید
- هر چیزی که کلاینت دریافت می‌کند قابل خواندن است — هیچ secretی در remote config نگذارید

تلهٔ مصاحبه

«*وقتی چیزی خراب شد، kill switch اضافه می‌کنیم.*» — باینری‌ای که روی میلیون‌ها دستگاه است هیچ switchی ندارد. حالا به hotfix، بازبینی و هفته‌ها صبر برای پذیرش نیاز دارید.

سؤال تکمیلی

«*Firebase یا سرویس خودمان؟*» — هر دو جواب می‌دهند. چیزی که مهم است: کش آخرین مقدار معتبر، پیش‌فرض امن، هدف‌گیری بر اساس نسخهٔ اپ، لاگ ممیزی از اینکه چه کسی چه چیزی را عوض کرد — و یک راه جایگزین سمت سرور که اصلاً به کلاینت وابسته نیست.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. «نسخهٔ 5.2 برای ۳ درصد کاربران کرش می‌کند. برش گردان.»

اهرم	سرعت	به چه کسی می‌رسد	محدودیت
خاموش کردن فلگ یا kill switch	چند دقیقه	دستگاه‌هایی که config را fetch می‌کنند	فقط کدی که پشت فلگ است
کاهش آسیب سمت سرور	چند دقیقه	همهٔ نسخه‌های اپ	فقط مسیرهایی که سرور کنترل می‌کند
halt کردن rollout مرحله‌ای	چند دقیقه	جلوی به‌روزرسانی‌های جدید را می‌گیرد	کاربران به‌روزشده باگ را نگه می‌دارند
رو به جلو — hotfix با versionCode بزرگ‌تر	چند ساعت تا چند روز	هر کسی که به‌روز کند	زمان بازبینی و پذیرش
هل دادن اصلاح — اولویت به‌روزرسانی و حداقل نسخه	بعد از انتشار اصلاح	کاربران فعال	به‌روزرسانی flexible قابل تعویق است

در incidentهای واقعی چند اهرم را با هم به کار می‌برید: همین حالا halt و خاموش کردن فلگ، بعد hotfix.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

نسخهٔ پارسال: *«اگر یک شاخص از آستانه عبور کند، نسخهٔ پایدار قبلی باید دوباره برای کاربران آسیب‌دیده deploy شود.»* — **در اندروید ممکن نیست.** دستگاه‌ها downgrade نمی‌کنند و versionCode تکراری پذیرفته نمی‌شود. «rollback» یعنی **کد قدیمی** را با یک **versionCode جدید و بزرگ‌تر** دوباره build کنید — این هم در واقع roll forward است.

تلهٔ مصاحبه

پارسال: *«پایپ‌لاین hotfix همهٔ مراحل غیرضروری را رد می‌کند.»* — رد کردن تست‌ها زیر فشار، شروع incident شمارهٔ دو است. گیت‌ها را نگه دارید؛ پایپ‌لاین عادی را آن‌قدر سریع کنید که همیشه اجرایشان کنید. hotfix یک diff کوچک است، نه میان‌بر.

چیزی را که برای دیباگ لازم دارید نگه دارید

AAB، `mapping.txt` و native symbolها را برای **هر** versionCode آرشیو کنید. کرشی از buildی که نمی‌توانید deobfuscate کنید، خواندنی نیست.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. secret بدون کلید بلندمدت

```yaml
publish:
  needs: bundle
  runs-on: ubuntu-latest
  environment: production   # required reviewers
  permissions:
    contents: read
    id-token: write         # job may request an OIDC token
  steps:
    - uses: actions/checkout@v4
    - uses: google-github-actions/auth@v2
      with:
        workload_identity_provider: ${{ vars.WIF_PROVIDER }}
        service_account: ${{ vars.PLAY_PUBLISHER_SA }}
    # publish step runs with short-lived credentials
```

- GitHub یک **توکن OIDC** صادر می‌کند که نام ریپو و environment در آن آمده — مقدار فیلد `sub` چیزی مثل `repo:org/app:environment:production` است
- **Workload Identity Federation** در Google Cloud فقط به همین subject اعتماد می‌کند و یک توکن کوتاه‌عمر برای service account برمی‌گرداند — کلید JSONی نیست که لو برود یا لازم باشد rotate شود



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. هر secret کجا زندگی می‌کند؟

secret	کجا	نکته
keystore آپلود و پسوردها	secretهای environment یا یک vault	پشتیبان آفلاین؛ هرگز در git
دسترسی انتشار در Play	اگر ابزارتان پشتیبانی کند Workload Identity؛ وگرنه کلید JSON در secretهای environment	در Play Console فقط برای **یک اپ** و فقط با دسترسی‌های لازم دعوت شود
توکن آپلود گزارش کرش	secret در CI	فقط زمان build
کلیدهای API داخل اپ	**secret نیستند** — داخل APK منتشر می‌شوند (روز ۲۲)	محدود به package name و اثر انگشت امضا

تلهٔ مصاحبه

«*GitHub secretها را mask می‌کند، پس لو نمی‌روند.*» — mask کردن فقط برای مقدار دقیق و به‌صورت best-effort است: مقداری که از base64 برگردانده یا تبدیل شده، بی‌مشکل چاپ می‌شود. و هر کدی داخل job — حتی یک پلاگین آلوده — می‌تواند متغیرهای محیطی را بخواند. jobهای انتشار را حداقلی و pinشده نگه دارید (روز ۲۳).



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

trunk-based development، release train، پایپ‌لاین انتشار با دسترس‌پذیری بالا



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. trunk-based development

	branchهای بلندمدت (GitFlow)	trunk-based
ادغام	در پایان — mergeهای بزرگ	PRهای کوچک به `main`، دست‌کم روزانه
کار نیمه‌تمام	پنهان در branchها	mergeشده و **پشت فلگ پنهان**
انتشار	mergeهای پشت‌سرهم از `develop` به `release` و بعد به `main`	یک release branch کوتاه‌عمر از `main` جدا می‌شود
ریسک	جهنم merge، غافلگیری دیرهنگام	CI سریع و انضباط در فلگ‌ها لازم است

- پژوهش DORA، trunk-based development را جزو قابلیت‌های تیم‌های تحویل پرعملکرد می‌داند

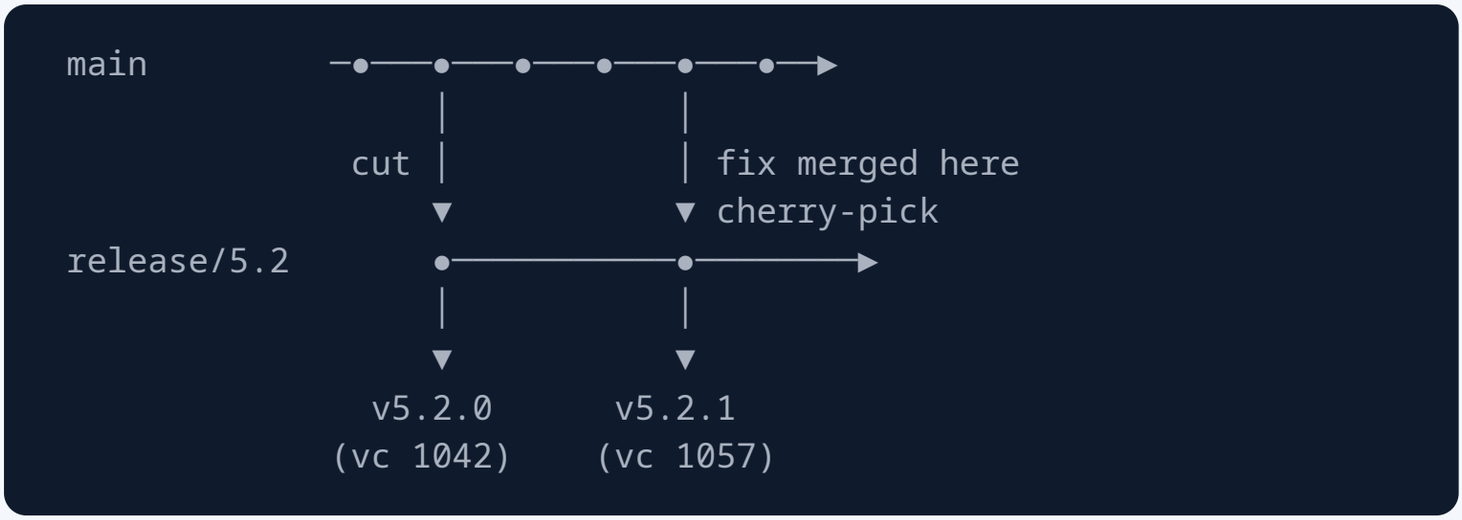
تلهٔ مصاحبه

«*trunk-based یعنی push به `main` بدون بازبینی.*» — نه. یعنی branchهای **کوتاه‌عمر**: چند ساعت تا یک روز، PR بازبینی‌شده، و گیت CI از روز ۲۳. نکته طول عمر branch است، نه حذف بازبینی.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. release branch: اول اصلاح روی main



- release branch **فقط اصلاح** می‌پذیرد — فیچر نه
- هر اصلاح اول روی `main` می‌نشیند، بعد با cherry-pick به release branch می‌رود
- هر build منتشرشده را تگ بزنید تا تگ، versionCode و فایل mapping به هم گره بخورند

تلهٔ مصاحبه

«*روی release branch درستش کن، بعداً merge می‌کنیم.*» — «بعداً» فراموش می‌شود و باگ در train بعدی برمی‌گردد. اول main، بعد cherry-pick به پایین.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. release train

```
Week   1          2          3          4
5.2    on main    cut, beta  rollout    100%
5.3               on main    cut, beta  rollout
5.4                          on main    cut, beta
```

- train طبق **برنامهٔ ثابت** حرکت می‌کند (هفتگی یا هر دو هفته)؛ branch در روز ثابتی جدا می‌شود
- فیچری که آماده نیست **از train جا می‌ماند** — پشت فلگش خاموش می‌ماند و با train بعدی منتشر می‌شود
- در هر لحظه: یک نسخه در حال rollout، یکی در بتا، و `main` در حال حرکت
- یک **release captain** چرخشی چک‌لیست را اجرا می‌کند و بر اساس داده دربارهٔ انتشار یا توقف تصمیم می‌گیرد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«*train را برای فیچر من نگه دارید.*» — یک تأخیر همهٔ تاریخ‌ها را قابل‌مذاکره می‌کند و trainها بزرگ‌تر و پرریسک‌تر می‌شوند. فیچر را خاموش منتشر کنید و بعداً روشنش کنید.

تلهٔ مصاحبه

«*train هفتگی یعنی کاربران روی آخرین نسخه‌اند.*» — پذیرش هفته‌ها عقب است. هم‌زمان از نسخه‌های زیادی پشتیبانی خواهید کرد: قرارداد API افزایشی، گیت سمت سرور بر اساس نسخهٔ اپ، و حداقل نسخهٔ پشتیبانی‌شده (روز ۲۹).

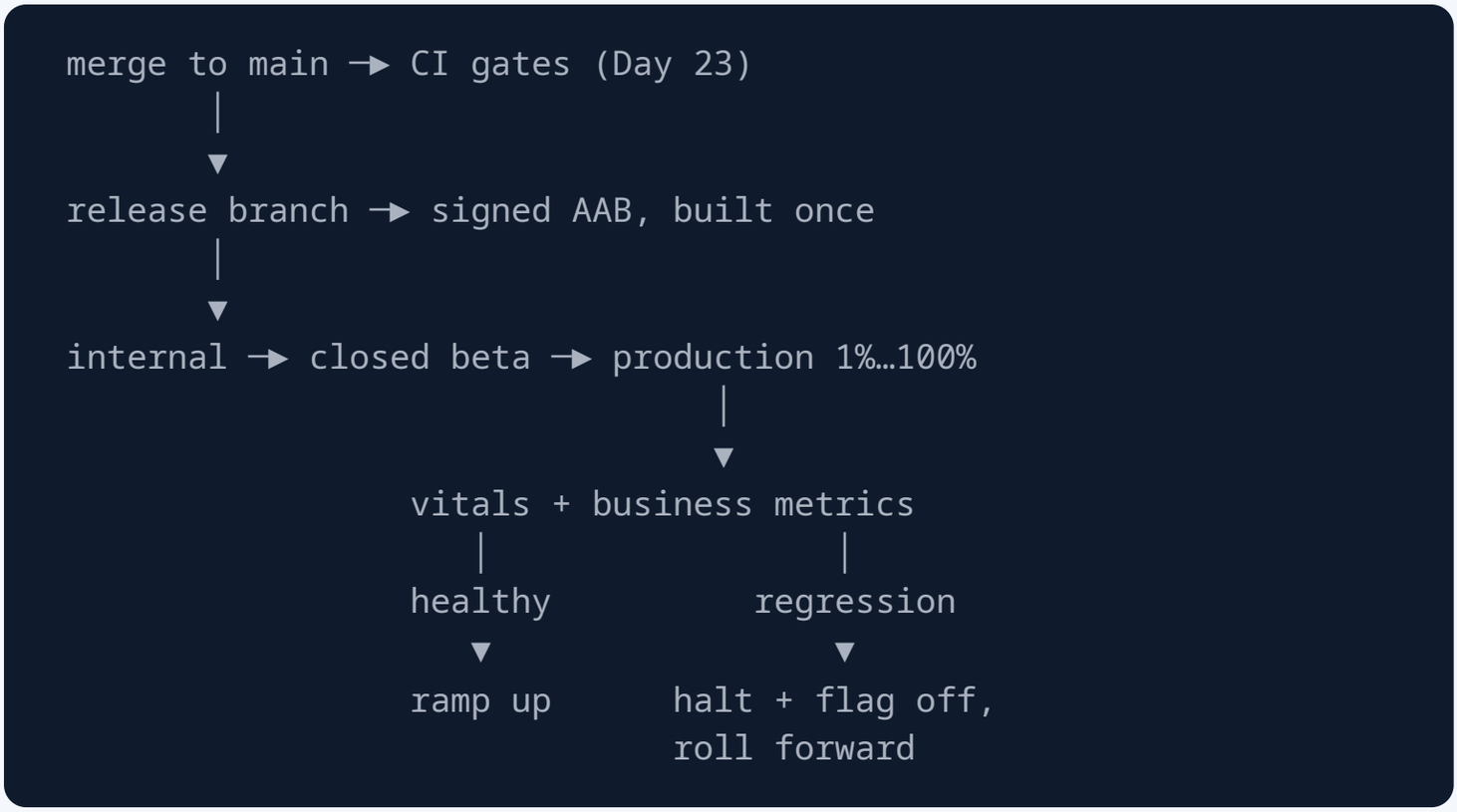
سؤال تکمیلی

«*train هر چند وقت یک بار حرکت کند؟*» — به همان اندازه که زمان بازبینی، ظرفیت QA و مانیتورینگ‌تان اجازه می‌دهد. train سریع‌تر یعنی انتشارهای کوچک‌تر، و جا ماندن از train هزینهٔ کمتری دارد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. طراحی انتشار برای یک اپ بانکی



تلهٔ مصاحبه

پارسال: «*استقرار blue-green برای اپ‌های موبایل.*» — نمی‌شود ترافیک را بین دو باینری نصب‌شده جابه‌جا کرد. معادل‌های موبایلی: rollout مرحله‌ای برای باینری، فلگ برای رفتار، و API نسخه‌بندی‌شده روی سرور.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. اصول پشت آن

اصل	چطور
نگهداری کلید	کلید امضای اپ نزد گوگل؛ کلید آپلود فقط در environment انتشار؛ انتشار با credentialهای کوتاه‌عمر
ردپذیری	تگ، versionCode، AAB و فایل mapping برای هر انتشار با هم آرشیو می‌شوند
تأیید انسانی	ارتقا به production بازبین لازم دارد؛ ترمز خودکار لازم ندارد
هر مسیر پرریسک یک switch دارد	فلگ با پیش‌فرض امن و مالک؛ سرور با همهٔ نسخه‌های پشتیبانی‌شده کار می‌کند
یک راه برای انتشار	hotfixها از همان پایپ‌لاین و همان گیت‌ها عبور می‌کنند
کف نسخه	حداقل نسخهٔ پشتیبانی‌شده و in-app update، کاربران را از buildهای ناامن جدا می‌کنند

جواب سینیور در یک جمله

«باینری منتشرشده را نمی‌شود پس گرفت: یک بار build کن، همان artifact را ارتقا بده، بر اساس داده بالا ببر، روی هر مسیر پرریسک ترمز بگذار — و از همان پایپ‌لاین رو به جلو درستش کن.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۲۴ — خلاصهٔ یک‌صفحه‌ای

- در Play App Signing کلید امضای اپ نزد گوگل است؛ **کلید آپلود** دست شماست و قابل ریست است.
- APIهای گوگل و App Links اثر انگشت **کلید امضای اپ** را می‌خواهند.
- versionCode فقط بالا می‌رود و مال CI است؛ versionName برای آدم‌هاست.
- AAB را یک بار بسازید و همان artifact را track به track ارتقا دهید.
- برای امضا در CI، keystore در secretهای environment است و Gradle متغیرهای محیطی را می‌خواند — هرگز در git.
- ابزار Gradle Play Publisher یا fastlane — هر دو با Play Developer API کار می‌کنند.
- halt کردن rollout جلوی گسترش را می‌گیرد، چیزی را برنمی‌گرداند. اول ترمز را خودکار کنید.
- in-app update با `app-update` ، به‌علاوهٔ گیت حداقل نسخهٔ خودتان.
- kill switch باید **قبل از** incident منتشر شده باشد. rollback نداریم: halt، خاموش کردن فلگ، roll forward.
- trunk-based و release train: اصلاح اول روی `main` ، بعد cherry-pick به پایین.

روز ۲۵ در راه است: ماژول‌بندی و Dynamic Feature. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700